Bagging, SAMME Boosting and Measured Bias–Variance with Raw-Pixel Decision Trees on Olivetti Faces

Om Dinesh Dhande
Roll No. M26CSE014
Repository: https://github.com/omdhande/Olivetti-Assignment2-M26CSE014
Final commit: **[run: git rev-parse HEAD]**

Abstract

We study decision-tree ensembles on the Olivetti faces (40 identities, 6/2/2 train/validation/test images per identity, raw 4096-pixel inputs, no dimensionality reduction). Bagging and multiclass SAMME boosting are implemented from scratch; trees come from scikit-learn. We (i) build a frozen single-tree baseline, (ii) analyse bagging through out-of-bag accuracy, tree disagreement and vote margin, (iii) examine SAMME's sample-weight dynamics, and (iv) *measure* bias^2 and variance over 30 within-identity resamples. Bagging lifts clean test accuracy from 0.60 (tree) to 0.925 and removes about 96% of the single tree's variance, but loses 35 points when the central 16×16 block is masked; the single tree never splits inside that block and is unaffected. SAMME with depth ≤ 4 trees reaches only 0.29 and its predictions collapse onto a few identities. Lowest variance is therefore *not* best accuracy.

1 Setup

Data and split. `fetch_olivetti_faces` gives 400 images (64×64, flattened to 4096). The RNG seed is the numeric part of the roll number M26CSE014, i.e. 26014 (`numpy.random.default_rng`). Inside each identity the ten images are permuted and split 6/2/2 into train (N=240), validation (80) and test (80); the indices are stored in `q1_split.npz` and reused by every experiment. Each test face therefore has an identity with exactly two test images, so a constant predictor scores 2.5%.

Masked test set. A second test set sets rows and columns 24–39 (a centred 16×16 block, 6.25% of the pixels) to zero. Models are trained on clean images only. One test face is 1.25 accuracy points, so differences of one or two faces are not meaningful.

Models. Single trees and weak learners are `DecisionTreeClassifier`; bootstrap sampling, voting, SAMME weights and all metrics are our own code. Hyper-parameters are chosen on validation accuracy only.

Table 1: Q1(a): single-tree grid (shaded row = frozen setting). "Single" = fraction of leaves with exactly one training face.

[run scripts/q1a_grid.py to generate this table]

Tables report one run with the stated seeds.

2 Q1: Tree baseline and bagging

2.1 Single-tree baseline (a)

We trained all 5×3 combinations of `max_depth` $\in \{2, 4, 8, 16, \text{None}\}$ and `min_samples_leaf` $\in \{1, 2, 4\}$ (Table 1). The setting with the strictly highest validation accuracy (ties broken by loop order) was frozen: **`max_depth=None, min_samples_leaf=4`**. Its measured properties are: validation accuracy **[run scripts/q1a_grid.py]**, test accuracy 0.600, training accuracy 0.967 (from the leaf class counts), 41 leaves, realised depth 31, 4–7 training faces per leaf (mean $240/41 = 5.85$), 36 of 41 leaves pure, and *no* leaf with a single training example (fraction 0).

What changes with depth. Depth d allows at most 2^d leaves, so depths 2 and 4 can emit at most 4 and 16 distinct identities for 40 classes; a depth-2 tree is capped at 10% accuracy (we measure 6.4% in Sec. 4). Deeper trees add leaves, shrink the mean number of faces per leaf ($= 240/\text{leaves}$) and push training accuracy towards 1. With `min_samples_leaf`$= 1$ a tree can isolate single faces; the constraint ≥ 4 forbids that, which is why the frozen tree has no singleton leaves even though its depth is unrestricted. The train/test gap (0.967 vs. 0.600) shows the unrestricted tree still memorises its six faces per identity: leaves are nearly pure but split on pixels that are not stable across images of the same person.

2.2 Bagging from scratch (b)

Each of $B_{\max}$=200 trees is fitted on a bootstrap sample of 240 faces drawn with replacement from the training set, using the frozen tree setting; smaller ensembles use the first B trees. Predictions are plurality votes (ties go to the lowest identity index). Table 2 gives train/validation/test accuracy and the mean number of distinct faces per bootstrap sample.

Distinct examples. A bootstrap sample of size n contains in expectation $n\big(1-(1-1/n)^n\big) = 151.89$ distinct faces for n=240 (63.3%, tending to $1-e^{-1}$). The count has standard deviation ≈ 4.8 per sample, so the mean over B trees has standard error $4.8/\sqrt{B}$. The observed averages (154.0 at B=1 down to 152.3 at B=200) are within 0.5 and 1.1 standard errors of the theory, i.e. consistent with it. About 36.7% of the training faces (≈ 88) are out of bag for any tree, which is why one bootstrap tree reaches only 0.658 training accuracy.

2.3 Ensemble diagnostics (c)

OOB accuracy votes, for each training face, only among trees whose bootstrap sample excluded it. *Disagreement* is the mean over tree pairs of the fraction of validation faces on which two trees differ. *Vote margin* is the winning vote fraction minus the runner-up's, averaged over validation faces. Figure 1 plots all three.

Final size. Validation accuracy ties at 0.8875 for B=100 and B=200 (one face separates B=50 from them), so we freeze the *smaller* ensemble, $B = 100$. Test accuracy was not used for this choice (B=200 would have scored 0.963 on test).

Do the three diagnostics agree? Not fully. OOB accuracy rises monotonically from $B\geq 5$ (0.28 $\rightarrow$ 0.76) and keeps rising after B=50, tracking validation accuracy. Disagreement is flat at ≈ 0.79 from B=5 and margin flat at ≈ 0.28 from B=25: they describe the *diversity* and *consensus strength* of the voters, which stabilise once the ensemble is large enough, whereas accuracy still gains. High disagreement (79% of faces) together with a one-tree validation accuracy of 0.363 means the trees are individually weak but make different errors, which is exactly the regime in which voting helps. A margin of 0.28 says the winner typically has only about 28 points more votes than the runner-up (plurality with 40 classes, not majority). OOB is pessimistic relative to validation (0.729 vs. 0.888 at B=100): each OOB prediction uses only $\approx$ 37% of the trees, and the held-out face is missing from every voting tree's training data, which for identities with six images is a $1/6$ loss.

2.4 Clean versus masked test (d)

Bagging corrects the tree on 27 clean faces; the tree is right and bagging wrong on one; both are wrong on five. The single tree is *identical* on clean and masked images: none of its 40 internal splits uses a pixel inside the masked block, so its predictions and confidences cannot change. In contrast 94 of the 100 bagged trees contain at least one split in the block (280 of 3976 splits, 7.0%). Bagging goes from 0.925 to 0.575: 28 faces change from correct to wrong and none from wrong to correct. The masked images look alike in the centre, and 33 of 80 masked predictions go to identity 13 (on clean images no identity receives more than 3). Mean winning vote falls from 0.366 to 0.267.

What bagging can and cannot fix. In the two corrected faces the tree is wrong with confidence 1.00 and 0.57 because one pure leaf, shaped by the particular six training faces and one greedy path, sends the face to the wrong identity; bagging recovers the right identity as a plurality (19 and 40 votes) because the other bootstrap trees take different paths. This is *variance*: the error depends on which faces a tree saw. In the two both-wrong faces the correct identity never leads. For face 18 it gets at most 10 votes, so most resampled trees share the same systematic confusion; for face 22 it ties with identity 8 at 13 votes and the lowest-index tie-break picks the wrong one. Errors that every resample reproduces (limited training views, raw-pixel features unrelated to identity) are *bias*-like and are not averaged away. Note also that tree "confidence" (leaf purity, ≈ 1) is high on wrong predictions, whereas vote fractions are low but informative.

3 Q2: Multiclass boosting (SAMME)

3.1 Algorithm and a worked round (a)

With K classes and sample weights w_i ($\sum_i w_i$=1), round t fits a tree h_t on weighted data, computes $\epsilon_t = \sum_i w_i \mathbf{1}[h_t(x_i) \neq y_i]$ and $\alpha_t = \log\frac{1-\epsilon_t}{\epsilon_t} + \log(K-1)$, then $w_i \leftarrow w_i \exp(\alpha_t \mathbf{1}[h_t(x_i) \neq y_i])$ followed by normalisation. Prediction is $\arg\max_k S_k(x)$ with $S_k = \sum_t \alpha_t \mathbf{1}[h_t(x) = k]$, and confidence is $q_k = S_k / \sum_t \alpha_t$. **Guard:** if $\epsilon_t \geq 1 - 1/K$ (no better than random guessing, so $\alpha_t \leq 0$) the learner is rejected and boosting stops; ϵ_t is clipped at 10^{-12} in the log and a perfect learner ends training. The face experiments use the same code path.

A useful identity: since $e^{\alpha_t} = \frac{(1-\epsilon_t)(K-1)}{\epsilon_t}$, the total weight of the misclassified set after normalisation is exactly $\frac{K-1}{K}$, the error of random guessing. SAMME hence forces the next learner to do better than chance on exactly the faces the last learner missed; a correctly classified face has its weight divided by $Z = (1-\epsilon_t)K$, a missed one multiplied by $\frac{K-1}{K\epsilon_t}$.

The stump splits between x=0.3 and 1.0 and labels the right side 2, so both class-1 samples are wrong. Their weight rises from $2/9$ to $2/3$ as predicted by the identity above.

Table 2: Q1(b,c): bagging versus ensemble size. Disagreement and margin are measured on the validation set; OOB on the training set.

B	Train	Val	Test	Distinct/sample	OOB acc.	Pairwise disagreement	Mean vote margin
1	0.658	0.363	0.350	154.0	0.314	0.000	1.000
5	0.917	0.525	0.600	153.0	0.284	0.823	0.203
10	0.983	0.713	0.638	151.0	0.402	0.798	0.241
25	1.000	0.838	0.875	152.0	0.588	0.786	0.268
50	1.000	0.875	0.938	151.3	0.667	0.791	0.284
100	1.000	0.888	0.925	151.9	0.729	0.793	0.280
200	1.000	0.888	0.963	152.3	0.763	0.796	0.281

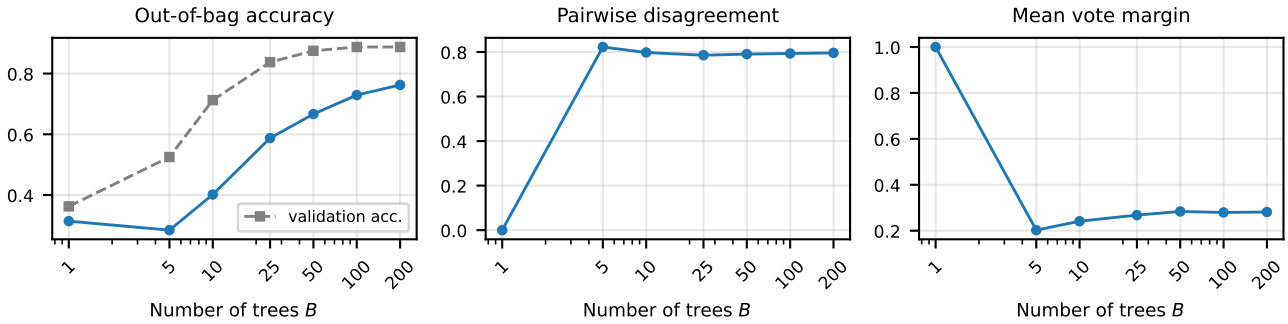


Figure 1: Q1(c): OOB accuracy (with validation accuracy dashed), average pairwise disagreement and mean vote margin versus B (log axis). At B=1 disagreement is 0 and margin is 1 by definition.

Table 3: Q1(d): accuracy and mean confidence (tree: leaf class fraction; bagging, B=100: winning vote fraction).

	Clean		Masked	
Model	Acc.	Conf.	Acc.	Conf.
Single tree	0.600	0.958	0.600	0.958
Bagging	0.925	0.366	0.575	0.267

Table 4: Q1(d): selected clean-test faces (test position / dataset index). Votes are of 100 bagged trees (top three).

Face	True	Tree (conf.)	Bagging votes	Outcome
1 / #3	0	23 (1.00)	0:19, 16:15, 28:15	bag fixes
3 / #11	1	15 (0.57)	1:40, 30:9, 26:9	bag fixes
18 / #98	9	0 (0.86)	28:17, 24:14, 39:11	both wrong
22 / #112	11	8 (1.00)	8:13, 11:13, 20:8	both wrong (tie)

3.2 Tuning on Olivetti (b)

Weak learners are `DecisionTreeClassifier` with `max_depth`$= d$, $d \in \{1, 2, 3, 4\}$, seeds $26014 + t$; round counts $10, 25, 50, 100$ are prefixes of one 100-round run per depth. Selection uses clean validation accuracy only (Table 6): **depth** 4**,** 100 **rounds** (validation 0.3375, train 0.8875,

Table 5: Q2(a): one complete SAMME round on 9 samples, K=3, depth-1 stump. $\epsilon = 2/9 = 0.2222$ (random-guess threshold $2/3$), $\alpha = \ln 3.5 + \ln 2 = 1.9459$. Unnormalised sum $= 2.3333$.

i	1	2	3	4	5	6	7	8	9
x_i	.0	.1	.2	.3	1.0	1.1	2.0	2.1	2.2
y_i	0	0	0	0	1	1	2	2	2
$h(x_i)$	0	0	0	0	**2**	**2**	2	2	2
w_i (init.)	0.1111 for all								
w_i (new)	.048	.048	.048	.048	**.333**	**.333**	.048	.048	.048

34.9 s). The best cell sits at the edge of the prescribed grid, so more rounds might still help.

Dynamics (Fig. 2). The weighted error starts at 0.875, falls to its minimum 0.377 in round 7 (where α peaks at 4.17), and then hovers around 0.66 (mean 0.662, range 0.38–0.875); it never approaches zero. This is structural: a depth-4 tree has at most 16 leaves and therefore at most 16 of 40 identities, so under near class-balanced weights its error cannot fall much below 0.6; and by the identity above the previous learner's mistakes always carry 97.5% of the mass. The learner weights stay in $[1.72, 4.17]$ (mean 2.98, $e^{\alpha_t} \approx 20$), i.e. almost uniform votes. Sample weights concentrate: the maximum weight peaks at 0.0857 (round 44, 20.6× uni-

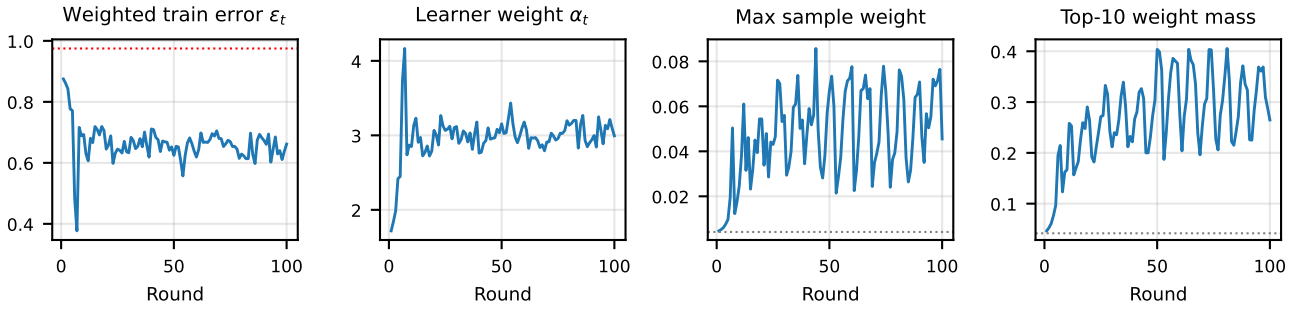


Figure 2: Q2(b), selected model (depth 4, 100 rounds): weighted training error (dotted red $= 1 - 1/K$), learner weight α_t, maximum normalised sample weight (dotted: uniform $1/240$) and total weight on the 10 heaviest faces (dotted: uniform $10/240$).

Table 6: Q2(b): SAMME train / validation accuracy.

Depth	10 rounds	25	50	100
1	.050/.038	.050/.025	.167/.125	.138/.075
2	.108/.088	.258/.200	.358/.175	.500/.225
3	.125/.100	.313/.138	.729/.250	.917/.288
4	.283/.125	.746/.250	.604/.250	.888/.338

Table 7: Q2(c): train/validation/clean-test accuracy and training time (s). Bagging uses the frozen unrestricted tree; boosting uses depth 4.

Model	Learners	Train	Val	Test	Time
Bagging	25	1.000	0.838	0.875	27.1
Boosting	25	0.746	0.250	0.263	8.9
Bagging	50	1.000	0.875	0.938	52.4
Boosting	50	0.604	0.250	0.250	17.8
Bagging	100	1.000	0.888	0.925	105.9
Boosting	100	0.888	0.338	0.288	35.7

form) and the ten heaviest faces hold up to 40.5% of the mass (round 81; uniform: 4.2%). Both curves oscillate with a period of roughly 10 rounds: a heavy group is fitted, its weight is divided by ≈ 14, and attention moves to whichever faces were missed. Training accuracy is therefore not monotone in the number of rounds (0.746 at 25, 0.604 at 50, 0.888 at 100).

3.3 Boosting versus bagging (c)

Bagging is far more accurate (0.925 vs. 0.288 at 100 learners) but about $3\times$ slower, because its unrestricted trees have ≈ 41 leaves and depth ≈ 31 whereas boosting's depth-4 trees have 8.9 leaves on average. The boosting train–test gap (0.888 vs. 0.288) shows that re-weighting memorises hard training faces rather than finding features that transfer.

A heavily weighted face. Replaying the run (the replay

[run scripts/q2c_weight_history.py]

Figure 3: Q2(c): the heaviest training face and its weight over rounds (red: rounds in which it is misclassified; dotted: uniform weight).

Table 8: Q2(d): accuracy and mean winning confidence for the three frozen models (bagging: vote fraction, B=100; boosting: $\max_k q_k$).

	Clean		Masked	
Model	Acc.	Conf.	Acc.	Conf.
Single tree	0.600	0.958	0.600	0.958
Bagging	0.925	0.366	0.575	0.267
Boosting	0.288	0.126	0.338	0.118

reproduces the stored α_t) shows how a single face keeps the learners' attention (Fig. 3). Training face #**[q2c]** (dataset index **[q2c]**, identity **[q2c]**) reaches weight **[q2c]** (**[q2c]**$\times$ uniform) in round **[q2c]** and is misclassified in **[q2c]** of 100 rounds, first in round **[q2c]**; its most frequent wrong label is identity **[q2c]**, and the final ensemble prediction for it is **[q2c]**. The mechanism follows from the update rule: when the face is missed its weight is multiplied by $\frac{K-1}{K\epsilon_t} \approx 1.5$ (so it stays heavy while the weights of the faces that were fitted are divided by ≈ 14), and a depth-4 tree that must also serve the other identities cannot isolate it, so the next learner is again pushed towards it. Of the other five images of its identity, **[q2c]** are among the ten heaviest faces at the end.

3.4 Clean versus masked test (d)

Boosting is the weakest model on both sets and its accuracy *rises* by 5 points under masking, but this is 11 faces going from wrong to right and 7 from right to wrong (51 of 80 predictions change), i.e. churn around a near-constant predictor,

Table 9: Q3(a): measured bias^2, variance and test accuracy over 30 refits (mean $\pm$ std).

Model	Bias^2	Variance	Test acc. (mean)	Std
Single tree	0.356	0.630	0.507	0.049
Bagging (100)	0.526	0.023	0.787	0.044
Boosting (d4, 100)	0.885	0.016	0.318	0.088

not robustness: on clean images identity 3 receives 43 of 80 predictions, and on masked images identity 12 receives 31. Bagging shows the same kind of collapse onto a single identity (13: 33 of 80) but at a much higher starting accuracy. Fewer boosting weak learners read the block than bagged trees do (50 of 100, 8.1% of 794 splits, 50.7% of the α mass, versus 94 of 100 trees).

A face that reacts differently. Test face 55 (dataset index 275, identity 27): bagging votes 27 with 38 of 100 votes on the clean image but only 19 votes on the masked one, losing to identity 13 (22 votes), so bagging flips from correct to wrong. Boosting predicts 27 on both images ($q_{27} = 0.178$ clean, 0.156 masked, runner-up identity 12 at 0.125) and stays correct. For bagging, the high tree disagreement (0.79) means that correct plurality rests on $\sim$40 votes spread across many partly independent trees; the mask changes the path of 94 trees at once, the votes diffuse, and 3 trees decide the outcome. For boosting, the weighting has concentrated sample weight on a few faces, so each learner is fitted to idiosyncratic detail and votes with a similar $\alpha \approx 3$; the mass is spread over many identities (mean winner 0.12–0.13) and the outcome is determined by a margin of $\approx$ 0.03. The face stays correct by a small margin rather than by greater robustness: in the same experiment boosting changes more predictions than bagging.

4 Q3: Bias and variance as measured quantities

Protocol. The clean test set is fixed. We draw R=30 new training sets by sampling, *within each identity*, six faces with replacement from that identity's six original training faces (expected distinct faces per identity: $6(1-(5/6)^6) = 3.99$), and refit the frozen tree, a 100-tree bagged ensemble and the selected SAMME (depth 4, 100 rounds) without re-tuning. For repetition r, $p_r(x_i) \in \mathbb{R}^{40}$ is the leaf class distribution (tree), the mean of tree class distributions (bagging) or $q(x_i)$ (boosting). Then Bias$^2 = \frac{1}{N}\sum_i \|\bar{p}(x_i) - e_i\|^2$ and Var $= \frac{1}{NR}\sum_{i,r}\|p_r(x_i) - \bar{p}(x_i)\|^2$ with N=80.

4.1 Selected models (a)

For the single tree the leaf distributions are (practically) one-hot, so Bias^2 + Var $= \mathbb{E}\|p_r - e\|^2 = 2(1 - \text{acc})$; this holds here ($0.356 + 0.630 = 0.987 = 2(1 - 0.507)$), which is a useful correctness check. The tree has the lowest bias^2 and a variance of 0.63 – about two thirds of the maximum 2 expected for hard predictions with accuracy $\approx$ 0.5. Bagging cuts variance by a factor of 27 ($0.630 \to 0.023$, 96%) and raises mean accuracy by 28 points. Resampling from six faces leaves only $\approx$ 4 distinct faces per identity, so the refit accuracies (0.507, 0.787) are below the full-data values (0.600, 0.925): these are bias and variance *for a reduced training set*. Boosting has the lowest variance and the highest bias^2 (0.885, close to the value 1 of a flat vector).

4.2 Effect of depth (b)

Going from depth 2 to unrestricted depth, single-tree bias^2 falls from 0.906 to 0.356 (-61%) while variance rises from 0.042 to 0.630 ($\times$15): the usual trade-off, with accuracy rising from 0.064 to 0.507. Bagging removes 0.041 of variance at depth 2 (98% of a small amount) and 0.607 at the selected depth (96% of a large amount); in both cases the residual variance is $\leq$ 0.024. The replicate row estimates seed noise: variance removed differs by 0.013 and accuracy gain by 1.5 points for the identical configuration, so the two unrestricted rows are not separable. (A finite intermediate depth, e.g. 8, is a straightforward extension of the same script.)

4.3 Is the deepest tree where bagging helps most? (c)

No. The largest accuracy gain from bagging occurs at the *shallowest* depth (+40.2 points against +28.0 and +29.5), while the largest variance reduction occurs at the deepest (0.607–0.620 against 0.041). The measurements explain why: (i) *Depth 2* trees have at most 4 leaves, hence at most 4 distinct predictions for 40 identities, giving 6.4% accuracy; their variance is already small (0.042) and bias^2 dominates (0.906). Averaging 100 such trees, each with different bootstrap splits, lets the argmax cover many identities, so accuracy jumps although measured bias^2 does not fall ($0.906 \to 0.924$): bias^2 (distance of $\bar{p}$ to the one-hot vector) and argmax accuracy are different quantities, and bagging here repairs a *representational* limit rather than reducing variance. (ii) At the selected depth the tree already has 41 leaves of 4–7 faces (Table 1); it has no singleton leaves, so "deepest" does not mean "memorises single faces" and the textbook picture of a deep high-variance tree applies only partly. Its variance is huge (0.630) and bagging removes 96% of it, but the starting accuracy is already 0.507, so there is less headroom; the ensemble ends at the best absolute accuracy (0.787). (iii) The measured disagreement of

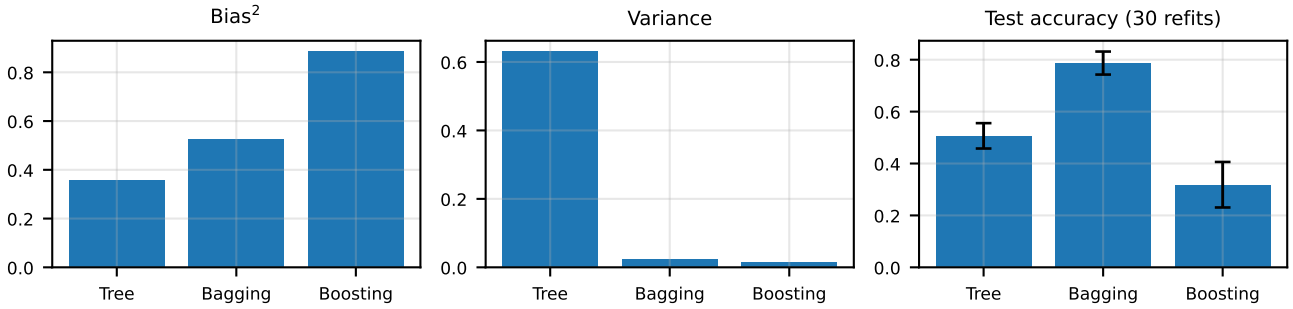


Figure 4: Q3(a): measured bias^2, variance, and mean test accuracy (error bars: std over 30 refits).

Table 10: Q3(b): depth sweep. The "selected" and "replicate" rows are the same configuration (unrestricted depth, `min_samples_leaf`$=4$) fitted with different random seeds; no deeper setting exists because the selected depth is already unrestricted.

	Single tree			Bagging (B=100)			Var. removed	Acc. gain	Var. removed
Depth	Bias^2	Var.	Acc.	Bias^2	Var.	Acc.	(absolute)	(points)	(fraction)
2 (shallow)	0.906	0.042	0.064	0.924	0.0009	0.466	0.041	+40.2	98%
None (selected)	0.356	0.630	0.507	0.526	0.0234	0.787	0.607	+28.0	96%
None (replicate)	0.377	0.643	0.490	0.528	0.0235	0.785	0.620	+29.5	96%

Table 11: Q3(d): clean/masked accuracy of the frozen models (original fit; bagging uses averaged class distributions) with bias^2 and variance from the 30 refits. With plain vote counts bagging scores 0.925 clean and 0.575 masked (Table 8).

Model	Clean acc.	Masked acc.	Bias^2	Variance
Single tree	0.600	0.600	0.356	0.630
Bagging	0.938	0.625	0.526	0.023
Boosting	0.288	0.338	0.885	0.016

bootstrap trees (0.79, Fig. 1) shows that diversity is high at the selected depth and constant in B, which is what makes the 96% variance removal possible. The residual error of the bagged model is mainly bias (0.526 vs. 0.023 variance), which bagging cannot reduce (it even rises from 0.356, partly because averaged class vectors are further from one-hot than hard predictions). So variance reduction is largest where trees are deepest, accuracy *gain* is largest where single trees are most biased and capacity-limited, and the best absolute accuracy is at the deep end.

4.4 Summary table and discussion (d)

Most stable. By measured variance boosting is lowest (0.016) and bagging close behind (0.023), a 27–38× reduction relative to the tree. But variance of class vectors is not scale-free: boosting's vectors are nearly flat (mean winning $q \approx 0.13$), which mechanically limits $\|p_r - \bar{p}\|^2$. By *accuracy* fluctuation, bagging (± 0.044) and the tree (± 0.049) are similar and boosting is the least stable (± 0.088), so boosting is stable in vector space but not in decisions.

Most sensitive to the missing region. Bagging: accuracy drops by 31 points (soft vote; 35 with hard votes), 28 faces go from correct to wrong, mean confidence falls from 0.366 to 0.267, and 94% of trees touch the block. The single tree is untouched because none of its 40 splits uses the block, but it is also the least accurate usable model. Boosting changes the most predictions (51 of 80) yet from a baseline near its own noise floor.

Does the lowest variance imply the best accuracy? No. Boosting has the lowest variance and the worst accuracy and highest bias^2; bagging has slightly higher variance but the best clean accuracy; the tree, with the highest variance, beats boosting. Variance is only one term: total error is Bias^2+Var (up to the noise term, here zero since labels are deterministic), and ranking models by variance alone ignores bias. The same holds for robustness: the most accurate model on clean data (bagging) is also the one that loses most under masking, because it exploits many pixels, including those in the block.

5 Limitations and reproducibility

Limitations. The test set has 80 faces (one face = 1.25 points); all tables use one split and one seed except Q3, which uses 30 resamples. The "deep" setting in Q3(b)

duplicates the selected setting. Bagging confidence differs between Q1/Q2 (hard votes) and Q3 (averaged class distributions), hence 0.925 vs. 0.938 clean accuracy. Boosting is limited by the prescribed weak-learner depth (≤ 4) and grid (≤ 100 rounds). Statements about mechanisms (e.g. why identity 3 or 13 act as default predictions) are consistent with the measurements but were not separately tested.

Reproduction. The repository contains the code, the saved split (`q1_split.npz`), final logs, figure scripts, `requirements.txt` and `README.md` with exact commands: `python Solution1.py`, `python Solution2.py`, `python Solution3.py` (in this order, from their folders), then `scripts/q1a_grid.py`, `scripts/q2c_weight_history.py`, `scripts/make_report_figures.py`, and `pdflatex main` in `report/`. Code commit: **`[run: git rev-parse HEAD]`**.

References

[1] L. Breiman. Bagging predictors. *Machine Learning*, 24(2):123–140, 1996.

[2] Y. Freund and R. E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *J. Comput. Syst. Sci.*, 55(1):119–139, 1997.

[3] J. Zhu, H. Zou, S. Rosset and T. Hastie. Multi-class AdaBoost. *Statistics and Its Interface*, 2(3):349–360, 2009.

[4] S. Geman, E. Bienenstock and R. Doursat. Neural networks and the bias/variance dilemma. *Neural Computation*, 4(1):1–58, 1992.

[5] F. S. Samaria and A. C. Harter. Parameterisation of a stochastic model for human face identification. In *WACV*, 1994.

[6] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *JMLR*, 12:2825–2830, 2011.